SOURCE CODE MANAGEMENT TOOL (GIT)

GIT REVERT (Safe Undo with History)

1. git revert creates a new commit that undoes the changes made by a previous commit.
2. It’s safe for shared branches because it doesn’t rewrite history — it adds a reversal commit.

CREATE A FILE

1. echo "hello" > file
2. git add .
3. git commit -m "v1"
4. git log --oneline

MODIFY THE FILE

1. echo "hey" >> file
2. git add .
3. git commit -m "v2"
4. git log --oneline

CHECK THE COMMITS

1. git log --oneline
   V1 -----> z9y8x7w v2
   V2 -----> a1b2c3d v1

I WANT TO ROLL-BACK FROM V2 TO V1

2. git revert a1b2c3d v1
3. git log --oneline
   v2 revert
   v2
   v1

CHECK THE FILE U ONLY GET THE V1 CONTENT

1. 1.cat file
   hello

GIT RESET

git reset is used to undo changes in the staging area (index) — that means it unstages files that were added using git add, without deleting or modifying the actual file content in your working directory.

2. echo "hello" > file
3. git add .
4. git reset file
5. git status

NOTE:

1. But if u run the { git reset --hard } git delete your file

---

GIT RESTORE

To update the file into the previous version if u made any wrong changes.

CREATE A FILE AND COMMIT

2. echo "new content" > file.txt
3. git add .
4. git commit -m "file.txt"
5. git push

ADD THE CONTENT INTO THE EXISTING-FILE

1. echo "hello" >> file.txt
2. git add .

RESTORE FILE FROM STAGING-AREA

1. git restore --staged file.txt
   or
2. git reset file.txt

RESTORE THE FILE TO PRIVIOUS VERSION

3. git restore file.txt
4. cat file.txt
   new content

GIT BRANCH

1. A Git branch is an independent line of development that allows you to work on a specific task without affecting other branches.
2. Each new feature, bug fix, or enhancement should be developed in a separate branch.
3. A new branch is created from an existing branch and initially contains the same code and commit history.
4. Branches enable multiple developers to work in parallel on the same repository without interfering with each other's changes.
5. After the work is completed and tested, the branch is merged into the main branch (main or master) to combine the changes.

BRANCHING STRATEGIES

Environments ----> testing team/ development team/ devops team

when a software is created ----> multiple stages/environments deployed

Flipkart ---------> developer created
Flipkart.com ---> clients can access the flipkart.com

Example:
Order something (grocery) ---> electronic items purchase ---> malfunctioning of software is called bug

Dev env
Developer deployed software

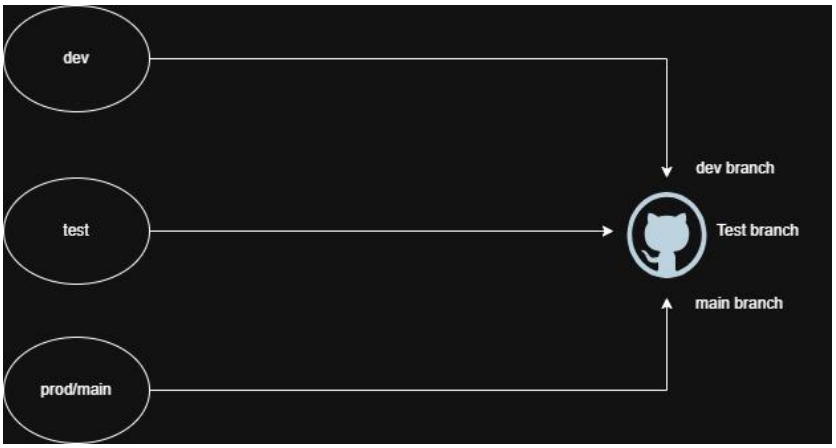
test env
test env server ---> test software deployed ----> test team test code with automation or manual using scripts

uat env (mirror env)
sale on Flipkart -----> ready with new code for sale -----> atleast 20 days before sale

prod env (code live)
ready code (deployed on prod env) ------> client-based user

1. developer develops new source code and deploys it in the dev server and soft final checking is done from dev once and then send it to the test env
2. testing team gets the new source code and then deploys it in the test server and then they do a Vigor testing on the newly created version code

3. In UAT environment your actual mirror application is ready and after few weeks it will be deployed in the prod environment but in this environment final review is been taken from the actual user and clients.
4. Once everything is finalised the code is deployed it into the prod environment server and clients can now use these new versions software as well.
5. Deployed of the new version software from one env to another env is done by the devops engineer.
6. So devops engineer having all the access of the servers while the dev/testing/uat having the specific server access.



How to create a branch

1. git branch ---------- to check available branches
2. git branch dev---------- to create a dev branch
3. git checkout dev--------to switch to the dev branch
4. git merge dev ---------- to merge the dev branch in master/main.
5. git branch -d dev------- to delete the branch

PRACTICAL

1. git branch dev
2. git checkout dev
3. vim file.txt
   Hello
4. git add .
5. git commit -m “v1-from-dev”
6. git push <repo_url> dev
7. git checkout main
8. git merge dev
9. ls

10. git branch test
11. git checkout test

12. git merge dev
    vim file1.txt
13. git add .
14. git commit -m “v1-from-test”
15. git push <repo_rul> test
16. git checkout main
17. git merge test
18. ls

BRANCHING BEST PRACTICES

1. one feature=one branch
   create a dedicated branch for each feature, bug fix or improvement

2. never develop directly on main
   protect the main branch and keeps it stable.
   All changes should go through pull requests.

3. Delete merged branches
   Once a branch is merged delete it to keep the repository clean

4. use meaningful branch names (feature/user-login)
   Use clear, descriptive names that indicate the purpose of the branch

5. keep branches short-lived
   Merge your changes as soon as possible to reduce risk and conflicts.

GIT IGNORE

git ignore is used to ignore specific files or folders so that Git does not track them or add them to the staging area.

1. touch app.py
2. touch .env
3. touch app.log
4. mkdir node_modules
5. touch node_modules/test.txt

6. git status

create a gitignore file

1. vim .gitignore
   .env
   *.log
   node_modules/

2. git status
   Only the app.py and .gitignore files are untracked files

3. git add app.py .gitignore
4. git commit -m "Add app and gitignore"
5. git status

GIT TAGS

Git Tags are used to mark important commits, such as software releases or versions.

Types of Git Tags

1. Annotated tags
   Annotated tags store additional information such as the tag name, message, creator, and date.

2. Lightweight tags
   Lightweight tags only point to a commit. They do not store extra information like a tag message or creator.

1. Annotated tags

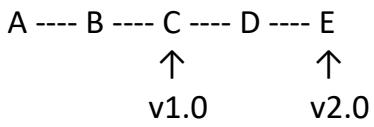
1. vim file.txt
   Hello
2. git add .
3. git commit -m “Release 1”
4. git tag -a v1.0 -m “My release 1”
5. git tag
   v1.0
6. git log –oneline
7. git show v1.0
8. git push origin v1.0

2.Lightweight tags

1. change in file.txt
   Hiii
2. git add .

3. git commit -m “Release 1.1”
4. git tag v1.1
5. git tag
   v1.0
   v1.1
6. git log –oneline
7. git show v1.1
8. git push origin v1.1

Example
Suppose your project history is:



- **Commit A** → Project created
- **Commit B** → Login feature
- **Commit C** → First stable release → **Tag: v1.0**
- **Commit D** → Payment feature
- **Commit E** → Second stable release → **Tag: v2.0**

GIT CLEAN

git clean basically used for removing the untracked files from the working directory.

TO CHECK WHICH FILES ARE DELETED
git clean -n

TO DELETE ALL UNTRACKED FILES
git clean -f

TO DELETE UNTRACKED DIRECTORIES
git clean -fd

TO DELETE IGNORED FILES
git clean -f -X

TO DELETE IGNORED + UNTRACKED FILES
git clean -f -x

TO CLEAN ONLY SPECIFIC FILES OR DIRECTORIES
1. git clean -f <filename>

2.git clean -fd <directory>

GIT CLONE

git clone is used to copy/download an entire repository from a remote location (like GitHub) to your local system.

1.git clone <repository-url>

GIT PULL

1. git pull is a combination of git fetch + git merge (or rebase), meaning it both downloads' changes and immediately integrates them into your current branch.
2. git pull is used to download changes from a remote repository and update your current branch automatically.

1.git pull origin main

GIT CONFLICTS

1. when same file having different content in different branches, if you do merge, conflicts occur.
2. conflicts occurs when you merge branches.

A. In master branch

1.vim file
hii
2.git add .
3.git commit -m "(message)"

B. In dev branch

1.vim file
hello
2.git add .
3.git commit -m "(message)"

A. Go to master branch

1.git checkout master

2.git merge dev
(merge conflict in file)
2.vim file
set the file according to you

GIT STASHING

1. suppose you are implementing a new feature for your product. your code is in progress and suddenly a customer escalation comes because of this, you have to keep aside your new feature work for few hours.
2. you cannot commit your partial code and also cannot throw away your changes.so you need some temporary storage when you can store your partial changes and later on commit it.
3. to stash an item (only applies to modified files not new files)

- git stash ---------- to stash all the files
- git stash push file1.txt --------- to stash a single file
- git stash list ---------- to show stash items
- git stash apply (stash @{0})------ to apply stash item
- git stash clear ---------- to clear the stash folder
- git stash pop ---------- to pull all stash items

WORKFLOW

1.CREATE A FILES
touch file1.txt file2.txt file3.txt

2.ADD THE FILE IN STAGING-AREA
git add .

3.COMMIT THE CHANGES
git commit -m “Added file1-file2-file3”

4.SUPPOSE MODIFIED THESE FILES
modified file1.txt
modified file2.txt
modified file2.txt

3.ADD THESE FILES IN THE STASHING-AREA
git stash

4.TO LIST ALL THE FILES IN STASHING AREA

git stash list

5.TO RETRIVE FILES FROM STASHING-AREA
git stash pop

GIT TRACK

Concept: Git does not track the empty folders
Solution: If u want to track the empty folders follows these steps

1. mkdir dir1
2. mkdir dir2
3. git status
   No files to commit

4. cd dir1
   touch .gitkeep

5. cd dir2
   touch .gitkeep

6. git status
   Untracked files

7. git add .
8. git commit -m “Added dir1 & dir2”
9. git push

DIFFERENCE BETWEEN GIT REVERT AND GIT RESTORE

1.GIT REVERT

1. git revert works on the repository level.
2. git revert creates a new commit where they show what we deleted/edit in the file or reverses the changes of a previous commit.

2.GIT RESTORE

1. git restore works on the working directory
2. git restore only undo the changes and update the file into the previous version.

DIFFERNCE BETWEEN THE GIT-FETCH & GIT-PULL

1.GIT PULL
git pull download only the code that should be added into the repo as well as
git pull is a combination of git fetch + git rebase + git merge

2.GIT FETCH
git fetch can download the update from the remote branch but i cannot directly merge into the working dir.

CHERRY-PICK

1. git cherry-pick copies a specific commit from one branch and applies it to another branch, without merging the whole branch.

2. Git cherry-pick copies a specific commit from one branch and applies it to another branch. It's used when you want only one commit—not the whole branch—like applying a hotfix created elsewhere

1.I Have Two Branches
git branch
1. main
2. feature

You Created Three Commits In Feature
1. Commit ID Message
   c1 Added login UI
   c2 Fixed login bug
   c3 Added logout feature

Then I Only Want One-Commit
The login bug fix (c2) is urgent and we want only this commit in main branch.

Switch To Main-Branch
1. git checkout main
2. git cherry-pick c2

- This will create a new commit on main
- The changes from commit c2 are applied
- No other commits (like UI or logout) are included

BEFORE CHERRY-PICK

1. main branch commits
   A --- B --- C (main)
2. feature-login commits
   D --- E(c2) --- F (feature-login)

AFTER CHERRY-PICK

3. main branch becomes
   A --- B --- C --- E' (main)
